

Solution Design Document

Loan Application Intake Automation, UiPath Implementation

Project	LoanIntake (Week 1, UiPath)
Author	Chris Williams
Date	April 2026
Version	1.0
Platform	UiPath Studio 26.0, Orchestrator Cloud
Library Version	LoanIntake.Library 1.0.4

1. Executive Summary

This document describes the design and implementation of an automated loan application intake process built on UiPath. The solution ingests inbound loan applications from a shared email inbox, extracts applicant data, validates supporting documents, enriches the application with customer profile and credit bureau information, and routes each application to one of five outcomes based on configurable business rules.

The architecture follows the standard dispatcher and performer pattern. A Loader process reads emails and writes structured queue items to Orchestrator. A Worker process, built on the Robotic Enterprise Framework, consumes queue items, applies business logic, and either completes the transaction automatically or escalates it to a human reviewer through Action Center. Reusable activities are packaged in a versioned Library and consumed by both processes through a private NuGet feed.

The design favors clear separation of concerns, configurable thresholds, and graceful exception handling. It demonstrates how rule based automation can absorb the high volume routine portion of a loan intake workload while preserving human judgement for material decisions.

2. Business Context and Objectives

Loan intake at a typical mid sized bank is bottlenecked by manual data entry. Loan officers spend significant time copying applicant details from email into the core banking system, pulling credit reports, and checking application completeness. This manual work is repetitive, error prone, and scales linearly with volume. It also creates inconsistent service times: small consumer loans wait in the same queue as multimillion dollar commercial requests.

The intake automation addresses this by handling the predictable cases with clear policy answers and presenting only the genuinely judgement based decisions to a human. Specifically:

- Reduce manual handling time for routine applications to near zero.
- Eliminate transcription errors between email and the core banking system.
- Apply credit and amount thresholds consistently and auditably.
- Pre gather all supporting context before a human reviewer opens the case.
- Surface and quantify exception rates by category for continuous improvement.

3. Solution Architecture

The solution comprises three deliverables: a Loader process, a Worker process, and a shared Library of reusable activities. They are connected by an Orchestrator queue and a private NuGet feed.

3.1 Component Overview

LoanIntake.Loader connects to the shared Gmail inbox, retrieves unread loan application emails, parses applicant fields using regular expressions, persists PDF attachments to a working folder, and dispatches one queue item per email to Orchestrator. The source email is marked as read on success so the next Loader run does not reprocess it.

LoanIntake.Worker is built on the Robotic Enterprise Framework. It consumes queue items one at a time, invokes Library activities to validate documents, look up customer records, and retrieve credit scores, then routes the application to one of five outcomes. The Worker writes the chosen decision to the queue item Progress field for downstream reporting, and creates Action Center tasks for high value cases that require human review.

LoanIntake.Library contains fourteen reusable activities organized into Documents and CoreBanking namespaces. The Documents namespace exposes ValidatePdfIsReadable. The CoreBanking namespace exposes Connect, Disconnect, LookupCustomerBySSN, GetCreditScore, and supporting helpers. The Library is published as a NuGet package and consumed by both Loader and Worker through a shared private feed.

The Orchestrator Queue, named Loan_Applications_Intake, is the integration point between the Loader and the Worker. It is encrypted at rest, configured with three retries on failure, ninety day retention for completed items, and one hundred eighty day retention for failed items. The queue acts as a durable buffer that decouples ingestion from processing.

Action Center hosts human approval tasks for high value loans. Form definitions are authored in the Studio Form Designer and stored alongside the Worker project. When a task is created, it displays loan details to the assigned reviewer and captures an approve or decline decision plus optional notes.

3.2 Data Flow

An applicant emails their loan application to a shared mailbox. The Loader runs on a schedule, pulls unread emails, and for each one extracts seven fields: applicant name, social security number, date of

birth, requested amount, loan purpose, annual income, and employer name. PDF attachments are saved to a per run working directory keyed by Loader run identifier and email message identifier. The Loader then writes a queue item to Orchestrator containing the parsed fields plus the attachment path and metadata identifiers.

The Worker is triggered separately, either on demand or on schedule. It calls Get Transaction Item, then runs the loan processing workflow on the returned item. The workflow extracts queue item fields, validates the PDF as readable, connects to the mock CoreBanking service, looks up the customer by social security number, retrieves a credit score, evaluates the routing rules, and writes the resulting decision to the queue item Progress field. For high value cases it also creates a Form Task in Action Center.

Decoupling the Loader from the Worker through the Orchestrator queue yields several practical benefits. The two processes can be scaled independently. Multiple Worker robots can drain a queue in parallel during volume spikes without changing the Loader. The queue itself provides durable buffering: if the Worker is offline for maintenance, items accumulate safely until processing resumes. Retry policies, item expiration, and access control are all configured at the queue level rather than embedded in the process logic. This is the dispatcher and performer pattern that the UiPath Robotic Enterprise Framework is designed around, and it generalizes well to any high volume intake scenario beyond loans.

3.3 Configuration and Assets

Runtime configuration is loaded into a dictionary at the start of each Worker job through the InitAllSettings workflow. Keys include the queue name, the queue folder, retry counts for queue interactions, the maximum number of consecutive system exceptions tolerated, and the message templates used for logging. Storing configuration in a single dictionary lets the rest of the workflow read settings by name rather than by magic constants. In a production deployment most of these values would be promoted to Orchestrator Assets so that operations teams can adjust them without redeploying the package. Two credentials are also stored as Assets in the current implementation: an SMTP password used by the Loader and a placeholder API key used by the mock CoreBanking client. Both are referenced by name from within the workflows so that secrets never appear in source XAML.

4. Routing Logic and Decision Rules

The Worker evaluates four rules in priority order. The first matching rule determines the outcome, with a fifth path reserved for cases that cannot be processed because the applicant is not a known customer.

ESCALATE_PENDING_APPROVAL fires when the requested amount exceeds five hundred thousand dollars. These applications are routed to Action Center for underwriting review regardless of credit score. The dollar threshold is high enough that bank policy requires a human signature even for the most creditworthy applicants, so this rule takes priority over all credit based logic.

AUTO_DECLINE fires when the applicant's credit score is below five hundred eighty. Applications in this category are automatically declined and a decline notification would be issued in production.

Sending these to a human reviewer adds no value because the policy answer is already clear, and removing them from the review queue lets loan officers spend time on cases where their judgement actually changes the outcome.

AUTO_APPROVE fires when the requested amount is at or below ten thousand dollars and the credit score is at least seven hundred twenty. These are small consumer loans to creditworthy applicants and the policy permits straight through processing. In production the Worker would hand these to the loan booking system directly with no human interaction.

MANUAL_REVIEW is the catch all for everything in the middle band. Mid size requests with mid range credit, where policy permits a loan officer to apply judgement based on factors that the automation cannot easily quantify. These items remain in the queue with a **MANUAL_REVIEW** progress marker and would be assigned to a loan officer review queue in production.

BUSINESS_EXCEPTION covers the case where the applicant cannot be located in core banking records. This is treated as a business rule exception rather than a system failure, since the underlying systems are functioning correctly and the data is well formed. The item is flagged for manual triage rather than retried, because retrying the same SSN against the same database will produce the same result.

Thresholds, including the high value cutoff, the auto decline credit floor, and the auto approve credit ceiling, are intentionally externalized in the Process workflow so that they can be tuned without rebuilding the Library. In a production deployment these would be moved to Orchestrator Assets so that policy changes do not require redeployment.

5. Exception Handling Strategy

The Worker distinguishes between three exception classes that require different responses, and the Library activities cooperate by classifying their own failures into the same taxonomy.

Business Rule Exceptions are raised when input data is valid but does not satisfy a business rule. The canonical example is a customer lookup that returns no match: the system is healthy, the data is well formed, but the application cannot proceed because the customer is unknown. The transaction is marked as a Business Exception in the queue and the Worker continues with the next item. No retry is attempted because retrying will produce the same result. These cases are appropriate for human triage but not for automated reprocessing.

Recoverable System Exceptions are raised when a transient external failure occurs, such as a credit bureau rate limit or a temporary network blip against the core banking service. Library activities classify these as recoverable and return a structured error rather than throwing. `Process.xml` then throws a `System.Exception`, which Orchestrator counts as an Application Exception and retries up to three times based on the queue retry policy. This pattern keeps the retry decision in the queue framework where it belongs, rather than scattered through the activity code.

Terminal System Exceptions are raised when a non recoverable failure occurs, such as authentication failure against the core banking service or a missing credential. Retrying these will not

help. The Worker increments its consecutive system exception counter and stops once the configured threshold of three is reached. This prevents the bot from chewing through an entire queue while a backing service is hard down, which would otherwise mark every item as failed and trigger downstream alerts unnecessarily.

6. Reusable Library Design

LoanIntake.Library packages fourteen activities organized into two namespaces. The Documents namespace contains PDF and document level validation logic. The CoreBanking namespace contains an idealized interface to the bank core: Connect, Disconnect, LookupCustomerBySSN, GetCreditScore, and supporting helpers. CoreBanking activities accept a session token to model a real authenticated session and return structured success indicators along with error type and error message fields.

The Library is published to a private NuGet feed at version 1.0.4. Both the Loader and Worker consume it as a fixed dependency. Versioning is deliberate: a fix in the Library that affects routing logic, such as the credit score lookup repair shipped in version 1.0.4, becomes a new package version that consumers update at their own pace. This is the same discipline a software team would apply to a shared services package and it allows business logic to evolve independently of the process orchestration.

Mock data is hardcoded inside the CoreBanking activities and dispatched by SSN prefix: 100 returns Sarah Mitchell with a high credit score, 200 returns James Chen with mid credit, 300 returns Michael Rodriguez with lower credit, 400 returns Patricia Williams as a high net worth profile, 700 returns Robert Hayes with subprime credit, and 999 returns customer not found. Codes 500 and 666 are reserved for recoverable and terminal system exceptions respectively. This allows the Worker to be exercised across all routing paths and exception classes without requiring a real banking integration.

Activities expose a consistent return contract. Each one writes out `out_Success`, `out_ErrorType`, and `out_ErrorMessage` in addition to its domain specific outputs. `ErrorType` is one of `None`, `Recoverable`, or `Terminal`, which lets the calling workflow decide what kind of exception to throw without inspecting error message strings. This contract makes the activities testable in isolation and decouples retry policy decisions from the activity implementation.

7. Human in the Loop, Action Center

When a loan exceeds 500,000 the Worker creates a Form Task in Action Center rather than making an automated decision. The form is assigned to a designated approver and contains seven read only fields populated from the application data: applicant name, customer name from CoreBanking, social security number, annual income, requested amount, loan purpose, and credit score. The approver adds optional notes and clicks either Approve Loan or Decline Loan. The form definition is stored as JSON in the Worker project and was authored in the Studio Form Designer to ensure schema correctness.

The integration uses a fire and forget pattern. The Worker creates the Form Task and continues processing other queue items. The approver reviews tasks asynchronously through the Action Center inbox. This design choice was deliberate: a synchronous wait would block the entire Worker thread until each high value case is reviewed, which does not match how high volume intake actually runs in production. In a fuller implementation a second process would poll for completed tasks and trigger the booking or decline workflow accordingly.

8. Observability and Reporting

Each transaction writes its final decision to the queue item Progress field. The Orchestrator queue dashboard therefore acts as a real time operations report: the Status column shows Successful or Failed at the queue framework level, while the Progress column shows the actual business outcome chosen by the Worker. Filtering the queue by Progress value returns counts of AUTO_APPROVE, AUTO_DECLINE, MANUAL_REVIEW, ESCALATE_PENDING_APPROVAL, and BUSINESS_EXCEPTION items processed across any given time window.

Detailed step level logging is emitted to Orchestrator throughout the Process workflow, including extracted field values, CoreBanking lookup results, credit scores, and the chosen routing decision. Logs are tagged with a business process name field so that they can be filtered across robot runs.

End to end testing was conducted with a Python script that sends five hundred simulated loan application emails to the shared inbox, distributed evenly across the five outcome categories. The script shuffles items so they arrive in mixed order, applies a delay between sends to respect SMTP rate limits, and reconnects periodically to avoid stale connection drops. With five hundred items in the queue, the Worker processes them sequentially in approximately three to four minutes, demonstrating that the architecture scales cleanly past trivial test volumes. The same approach can be used to seed regression tests after Library updates or routing rule changes.

9. Known Limitations and Future Work

- PDF validation thresholds are currently configured in warning only mode. Documents that fail readability checks emit a warning but proceed to credit checks. Threshold tuning requires a sample document corpus from production.
- CoreBanking and credit bureau integrations are mocked through SSN prefix lookups. Production deployment requires real API integration and credential rotation through a secrets manager.
- Routing thresholds are hardcoded in Process.xml. These should be moved to Orchestrator Assets so that policy changes do not require a redeployment.
- The Action Center handoff is fire and forget. A companion process to consume completed Form Tasks and execute downstream booking or decline workflows is out of scope for the current iteration.

- Email parsing relies on regular expressions against a fixed template. Real applicant emails would require a more flexible extraction approach, such as Document Understanding, to handle varied formatting.
- The current configuration assumes a single processing folder, Shared. A production multi tenant deployment would partition queues, catalogs, and credentials by line of business.

10. Implementation Notes and Lessons Learned

Several non obvious lessons surfaced during implementation that are worth recording for future iterations and for teams adopting similar patterns.

10.1 String Switch Keys in XAML

Studio's visual designer wraps string keys in literal quote characters when authoring a Switch activity. The resulting XAML key becomes the five character string with surrounding quotes rather than the three character intended value. At runtime, the Switch evaluates the expression to a plain string and never matches any case, so every input falls through to the default. This bug surfaced first in LookupCustomerBySSN and again in GetCreditScore. The fix in both cases was to edit the XAML directly and remove the surrounding quote characters from each Switch case key. The general lesson is that declarative formats authored through visual tooling sometimes hide syntactic gotchas that the runtime then surfaces as silent misbehavior.

10.2 Configuration Dictionary Population

The standard Robotic Enterprise Framework template populates its configuration dictionary using an InvokeMethod activity calling Dictionary.Add. In practice this approach compiled cleanly in Studio but did not actually add entries at runtime in the current Studio version. Every key lookup later in the framework then raised KeyNotFoundException. The working pattern is to use Assign activities with the dictionary indexer syntax: out_Config of key name equals CObj of value. The CObj wrapping is required because the dictionary value type is Object and bare literals fail validation.

10.3 Library Activity Invocation in XAML

Library workflows consumed from a NuGet dependency are invoked as typed activity elements rather than through InvokeWorkflowFile. This requires both an xmlns alias declaration referencing the assembly and the package namespace, and a corresponding AssemblyReference entry. The package namespace replaces dots in the project name with underscores while the assembly name retains the dots. Getting either wrong yields an UnknownType validation error that is difficult to diagnose visually. Once the namespace declarations are correct, activities behave like any built in activity and their input and output arguments bind cleanly to workflow variables.

10.4 Action Center Form Schema

Action Center expects form definitions in a specific JSON schema that matches what the Studio Form Designer produces. Hand authored JSON based on older examples may parse but then fail validation

inside the activity. Authoring forms through the Form Designer is considerably faster than iterating through schema errors. The designer also makes the field key names visible, which is essential since those keys must match the keys passed into the FormData dictionary from the calling workflow.

11. Closing

The intake automation demonstrates the dispatcher and performer architecture, REFramework based exception classification, Library driven reusability, and Action Center based human handoff in a single cohesive solution. It absorbs the high volume routine portion of loan intake while preserving and structuring the genuinely judgement based cases for human review. With threshold tuning and live integrations, the same architecture is suitable for production deployment.